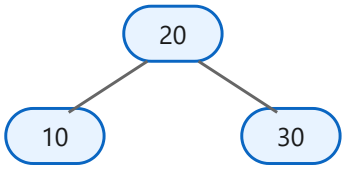
# Maps, Sets and Hashing

Lookup by key: `map` and `set` are sorted trees, the unordered containers are hash tables. `operator[]` versus `find`, `try_emplace`, `multimap`, custom comparators and hashes, and `pair` and `tuple`. Assumes STL Containers.

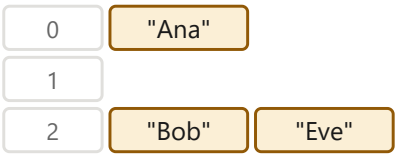
## Associative and unordered containers

**map / set: balanced tree, sorted, O(log n)**




in-order walk = sorted order

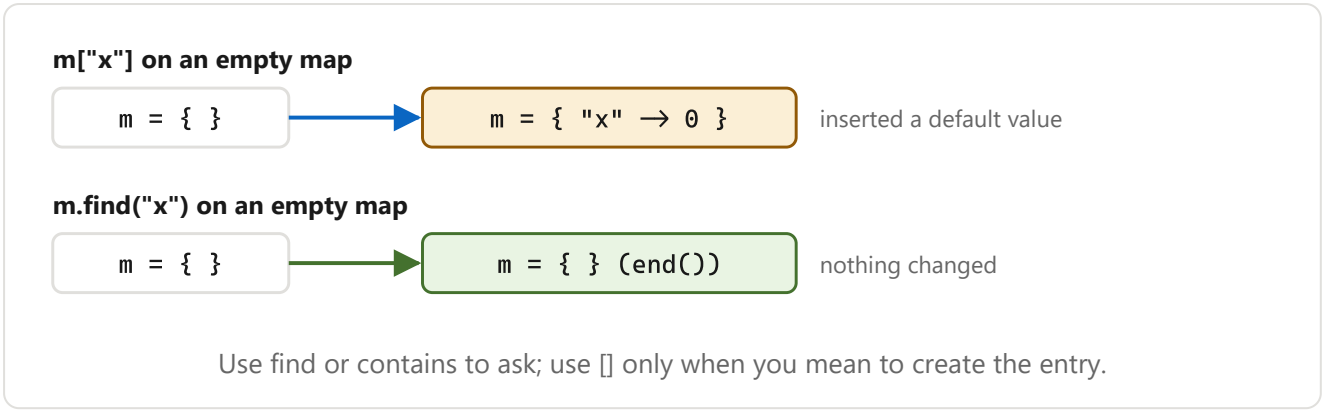
**unordered_map: hash table, average O(1)**




hash(key) picks a bucket; collisions chain (Bob, Eve)

Need order or range queries: map. Plain lookup: unordered_map.

| | `MAP` / `SET` | `UNORDERED_MAP` / `UNORDERED_SET` |
|---|---|---|
| Implementation | balanced binary tree (usually red-black) | hash table |
| Order of iteration | **sorted** by key | unspecified |
| find / insert / erase | O(log n) | average O(1), worst case O(n) |
| Key requirement | `operator<` (a strict weak ordering) | a hash function and `operator==` |
| Range queries (`lower_bound`) | yes | no |




Use find or contains to ask; use [] only when you mean to create the entry.

```cpp
std::map<std::string, int> age;
// operator[] INSERTS a default (0) if the key is missing
age["Ana"] = 30;
// does nothing if "Bob" already exists
age.insert({"Bob", 25});
age.insert_or_assign("Bob", 26);          // C++17: insert or overwrite

if (auto it = age.find("Ana"); it ≠ age.end())   // find never inserts
    // it→first is the key, it→second the value
    std::cout << it→second;

// structured bindings; sorted by name
for (const auto& [name, years] : age)
    std::cout << name << ' ' << years << '\n';

std::unordered_set<int> seen;
if (!seen.insert(5).second) { /* second == false means it was already there
*/ }
```

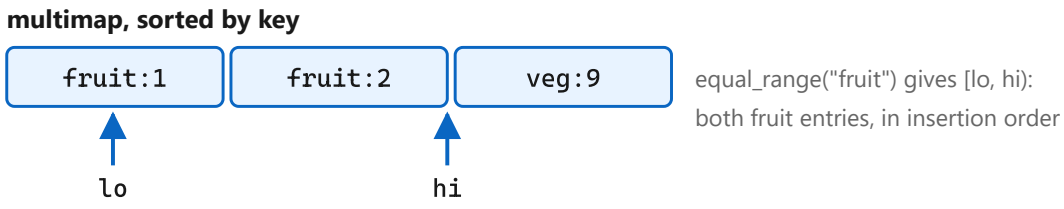
## `try_emplace`, `multimap` and `multiset`

| CALL | IF THE KEY ALREADY EXISTS |
|---|---|
| `m[k] = v` | overwrites (and inserts a default first if missing) |
| `m.insert({k, v})` / `m.emplace(k, v)` | does nothing, but **may already have moved from or built the value** |
| `m.try_emplace(k, args ... )` (C++17) | does nothing and **guarantees the arguments are not touched**: compiled, a moved-in 40-character string was intact afterwards |
| `m.insert_or_assign(k, v)` (C++17) | overwrites |

```cpp
std::map<int, std::string> a{{1, "old"}};
std::string payload(40, 'p');
// key 1 exists: nothing happens, payload still holds 40 chars
a.try_emplace(1, std::move(payload));
// new key: the string is moved into the map
a.try_emplace(2, std::move(payload));

// duplicate keys allowed
std::multimap<std::string, int> mm{{"fruit", 1}, {"fruit", 2}, {"veg", 9}};
// the range of all entries with that key
auto [lo, hi] = mm.equal_range("fruit");
// 2: there is no operator[] because a key may have many values
mm.count("fruit");
```

- `multimap` and `multiset` allow duplicate keys and keep equal keys in insertion order. There is no `operator[]`.
- `try_emplace` is the right call for "insert if absent" when building the value is expensive or the argument is move-only.




A multimap keeps duplicate keys next to each other.

---

## Custom keys: comparators and hashes

A `map` / `set` needs an **ordering**; an `unordered_map` / `unordered_set` needs a **hash and equality**. For your own key types, you provide them.

```cpp
// an ordering that ignores case
struct NoCase {
    bool operator()(const std::string& a, const std::string& b) const {
        return std::lexicographical_compare(a.begin(), a.end(), b.begin(),
b.end(),
            [](unsigned char x, unsigned char y) { return std::tolower(x) <
std::tolower(y); });
    }
};
```

```cpp
// the comparator is part of the TYPE
std::map<std::string, int, NoCase> m;
// the same key: size is 1 (compiled: 2 keys with "beta")
m["Alpha"] = 1; m["ALPHA"] = 2;

struct Product { std::string shop; int id; bool operator==(const Product&)
const = default; };
struct ProductHash {
    std::size_t operator()(const Product& p) const {
        std::size_t h = std::hash<std::string>{}(p.shop);
        // combine the member hashes
        h ^= std::hash<int>{}(p.id) + 0x9e3779b97f4a7c15ULL + (h << 6) + (h
>> 2);
        return h;
    }
};
std::unordered_map<Product, double, ProductHash> price;
price[{"north", 7}] = 101.5;
```

| CONTAINER | KEY MUST PROVIDE | RULE |
|---|---|---|
| `map`, `set` | a **strict weak ordering** (`operator<` or a comparator) | irreflexive and transitive; equivalence means "neither is less". `≤` is **not** valid: undefined behaviour |
| `unordered_map`, `unordered_set` | a hash and `operator==` | equal keys **must** have equal hashes; a poor hash makes every lookup O(n) |

- **Transparent comparators:** `std::map<std::string, int, std::less<>>` lets `find` take a `string_view` or `const char*` without building a temporary `std::string` (compiled: it found `"apple"` from a `string_view`). C++20 adds the same for unordered containers.
- **Never change a key** while it is in the container: its position or bucket would be wrong. Keys are `const` for that reason.
- **Hash quality:** combine *all* fields; avoid returning just one field. `std::hash` for `int` is the identity in libstdc++, which is fine.
- A `rehash` (growth beyond `max_load_factor`) invalidates iterators; `reserve(n)` up front avoids it, and avoids the allocations on a hot path.

- Ordered or hashed? Ordered when you need sorted iteration or range queries (`lower_bound`); hashed for plain lookup. For a few dozen keys a sorted `vector` and binary search often beats both.

> KEY POINT "How do you use your own type as a key in `unordered_map`?" Provide `operator==` and a hash functor (or specialise `std::hash`), combining all members, and make sure equal objects hash equally.

---

## `pair`, `tuple` and structured bindings




map::insert returns pair<iterator, bool>: structured bindings read it as [it, inserted].

A pair returns two values without a throwaway struct.

`std::pair<A, B>` holds two values, `std::tuple<A, B, C ...>` any number. Structured bindings (C++17) unpack them into named variables.

```cpp
// returns {smallest, largest}
std::pair<int, int> minmax(const std::vector<int>& v);
// lo = 1, hi = 9: two names instead of .first / .second
auto [lo, hi] = minmax({3, 9, 1, 7});

auto t = std::make_tuple(1, 2.5, std::string("x"));
// 2.5 (index must be a compile-time constant)
std::get<1>(t);
// "x" (or by type, when the type appears once)
std::get<std::string>(t);

std::map<std::string, int> m{{"a", 1}};
// insert returns pair<iterator, bool>
auto [it, inserted] = m.insert({"a", 2});
// inserted == false, it->second == 1: the key existed, nothing changed
```

- A pair or tuple has **no names**: `.first` and `std::get<2>` say nothing about meaning. For a result that leaves a function and is used in more than one place, a small named `struct` reads better.
- Comparison is built in and lexicographic: pairs and tuples compare element by element, so they work as `map` keys and in `std::sort`.
- `std::tie(a, b) = f();` assigns into existing variables; `std::ignore` skips one.
- Structured bindings also unpack plain structs and arrays: `auto [x, y] = point;`. They are copies unless you write `auto&` or `const auto&`.

## Further reading by section

Where each section of this lesson is explained in more depth. Pages are PDF viewer pages (the number your PDF reader shows).

| SECTION | BOOK | PART OF THE BOOK | PDF PAGES |
|---|---|---|---|
| **Associative and unordered containers** | C++ Primer | 11.1 to 11.2 Using an associative container, overview | 449 to 456 |
| | C++ Primer | 11.3 Operations on associative containers | 457 to 471 |
| | Programming: Principles and Practice | 20.1 to 20.6 map, unordered_map, set, container overview | 591 to 609 |
| **Custom keys: comparators and hashes** | C++ Primer | 11.3 Operations on associative containers | 457 to 471 |
| | C++ Primer | 11.4 The unordered containers | 472 to 477 |
| **pair, tuple and structured bindings** | C++ Primer | 17.1 The tuple type | 747 to 751 |